

Session-Based Chat System with LLM-Assisted State Tracking and Monitoring

Final Project Proposal

Yukuo Li

Objective

This project aims to build a session-based chat system that demonstrates object-oriented software design through user management, session management, structured conversation tracking, and backend monitoring. The system will organize and maintain chat interactions as persistent software objects rather than treating the application as a simple one-time message interface.

A language model API will be integrated as an assistance component for response generation and lightweight state extraction. However, the main focus of the project is not the language model itself, but the software system that manages sessions, stores structured interaction data, and coordinates the conversation workflow through object-oriented design.

System Description

After logging into the system, each user will enter a personal chat workspace. Users can create and manage multiple chat sessions, and each session will maintain its own conversation history and session-level state. Messages will be stored as part of a structured session record rather than as temporary outputs.

When a user sends a message, the system will process the message together with the current session context. An external language model API may be used to generate a more natural response and extract lightweight session information, such as the current topic or a short summary of the user's latest concern. This extracted information will then be stored and updated by the system as part of the session state.

In addition, the system will include a logging and monitoring component that records important interaction data, including messages, timestamps, session identifiers, and state updates. This allows the application to preserve a structured record of conversation progress and support a backend monitoring view.

To ensure that the project remains functional as a software system even when the API is unavailable, the application will also include a basic fallback response mechanism so that core workflows such as session creation, message storage, logging, and state updates can still be demonstrated.

Core Features

The main features of the system include:

- User registration, login, and logout
- Creation and management of multiple chat sessions
- Conversation history storage for each session
- Session-level state tracking and updates

- API-assisted response generation
- API-assisted lightweight state extraction
- Logging of messages, timestamps, and state records
- Backend monitoring of active sessions and latest status
- Persistent local storage of users, sessions, and logs
- Basic fallback handling when the API is unavailable

Technical Design

The project will be implemented in Python using object-oriented programming principles. The system will be structured into classes such as `User`, `ChatSession`, `Message`, `SessionState`, `ApiClient`, `ResponseService`, `StateTracker`, `LogManager`, `StorageManager`, and `AdminMonitor`. Each class will be responsible for a specific part of the system, such as user management, session handling, message representation, state storage, API communication, logging, or backend observation.

This design separates system responsibilities clearly and allows the project to emphasize modularity, maintainability, and extensibility. UML diagrams, including use case, class, and sequence diagrams, will be used to illustrate the system structure and interactions.

Expected Outcome

The final result will be a functional chat application that demonstrates structured session management, persistent data storage, session-aware interaction tracking, and backend monitoring within a clear object-oriented architecture. Although an external language model API may improve the quality of responses and state extraction, the primary outcome of the project is a software system that organizes and manages conversations in a structured and maintainable way.

The project is expected to show how object-oriented design can be used to integrate external intelligent services into a complete application while keeping the core system workflow, data organization, and monitoring logic under the developer's own control.